

WC Bill Review Agent - README

WC Bill Review & Code Anomaly Detection Agent

Portfolio project: an AI system that reviews **100% synthetic** workers' comp medical bills against real, public coding rules (CPT/HCPCS, ICD-10-CM, NCCI PTP edits, CMS MUE tables) and flags likely coding problems — unbundling, fragmented/excessive billing, diagnosis-procedure causality mismatches, and up-coding.

No real claims, bills, providers, or patients appear anywhere in this project. See `data-generation/DATA_CARD.md` for exactly what's real (the coding rules) vs. generated (everything else).

Status

- ☒ **Week 1** — reference rule engine + synthetic bill generator (`data-generation/`)
- ☒ **Week 2** — feature engineering + anomaly detection models (`models/`)
- ☒ **Week 3** — MLflow registry + drift detection (`monitoring/`)
- ☒ **Week 4** — RAG explanation agent (`agent/`)
- [~] **Week 5** — fine-tuned narrative generator – **deliberately cut**. Scoped as a multi-violation case-narrative generator, fine-tuned on a synthetic template-generated training set. Not built: Azure OpenAI fine-tuning has real training-job and ongoing hourly deployment-hosting costs, and the models that currently support fine-tuning (GPT-4.1-mini/nano, GPT-4o-mini – not the GPT-5.6 family already deployed for Week 4) would have needed a second resource/region. Cut as a cost/scope decision, not a technical one – see Week 4's RAG explainer for the fine-tuning alternative (retrieval + prompting) this project uses instead.
- ☒ **Week 6 (partial)** — FastAPI service + Docker (`api/` , `Dockerfile`) — **Azure deployment not done**. See `docs/DEMO.md` for a walkthrough.

Setup

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Running it end to end

```
# 1. Build the reference rule tables + database
cd data-generation
python3 reference/build_reference_data.py
python3 reference/load_reference_db.py

# 2. Generate the synthetic labeled bill corpus
python3 generate_bills.py --n-bills 4000 --seed 42

# 3. Build features, train both models, evaluate (now also logs to MLflow)
cd ../models
python3 features.py
python3 train_baseline.py
python3 train_xgboost.py
python3 evaluate.py
python3 ablation.py

# 4. Simulate a "current" batch of bills 6+ months later, check for drift,
#    and get a go/no-go retraining decision
cd ../data-generation
python3 generate_bills.py --n-bills 600 --seed 99 --dos-offset-max 320 --out-subdir drift_batch
cd ../monitoring
python3 drift_detection.py
python3 retrain_trigger.py    # exit code 1 == retrain recommended

# optional: browse the MLflow UI (run from the project root)
cd ..
```

```

mlflow ui --backend-store-uri sqlite:///mlruns.db
# then open http://127.0.0.1:5000

# 5. RAG explanation agent -- needs agent/.env filled in first, see
#   agent/RAG_AGENT.md's Setup section
cd agent
python3 build_index.py
python3 retrieve.py
python3 explain.py

```

Every step is deterministic (fixed seeds) — running it again reproduces the same numbers.

Where to start reading

- [data-generation/DATA_CARD.md](#) — what's synthetic, what's real, how the bill generator and violation injection work.
- [models/MODEL_FRAMING.md](#) — why multi-label, four binary classifiers.
- [models/EVALUATION.md](#) — the model results, and an honest look at what the numbers actually mean (short version: the first evaluation is too easy — read this before trusting a 1.0 precision/recall).
- [monitoring/DRIFT_MONITORING.md](#) — why this project needs drift monitoring at all (real CMS/AMA code-set update cycles), how the PSI / KS-test check works, and an honest read of what it did and didn't correctly catch on the demo batch.
- [agent/RAG_AGENT.md](#) — how the explanation agent retrieves the right rule for a flagged bill (two different retrieval strategies, and why), why embeddings are local but generation is hosted, and what's been verified with real runs (FastEmbed, retrieval, and Azure OpenAI generation, all confirmed working end to end).
- [docs/DEMO.md](#) — a five-minute walkthrough of the running app: how to start it (Docker or plain [uvicorn](#)), example bill IDs for each violation type, and what each part of the UI's response means.
- [docs/DEMO_VIDEO_SCRIPT.md](#) — a scene-by-scene narration script for recording a ~2.5-minute demo video of the app.

Project layout

data-generation/	Week 1 — reference data, bill generator, generated corpus
models/	Week 2 — features, training, evaluation, MLflow logging
monitoring/	Week 3 — drift detection, retraining trigger
agent/	Week 4 — RAG explanation agent (LangChain + Azure OpenAI)
api/	Week 6 — FastAPI service + static chat UI
Dockerfile	Week 6 — container packaging (model + vector store baked in)
infra/	Week 6 — Bicep + GitHub Actions (not started)
docs/	Demo walkthrough + screenshots